

Experiment ‘p_13_10_recursive_area_polygon_array’ Results

June 5, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 4

Problem Specification

Problem name: p_13_10_recursive_area_polygon_array

Natural language statement: null

Method signature: p_13_10_recursive_area_polygon_array(points: array<(real, real)>) returns (area: real)

Ensures

- `area == polygon_area(points[..])`

Requires

- `points.Length >= 3`

Functional Code Given

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
```

```
var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);  
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;  
triangle_area + polygon_area([points[0]] + points[2..])  
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

`p_13_10_recursive_area_polygon_array(points: array<(real, real)>) returns (area: real)`

The method should respect the following contract:

`requires points.Length >= 3, ensures area == polygon_area(points[..])`

The contract uses the following dafny code:

```
function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;
    triangle_area + polygon_area([points[0]] + points[2..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that

- surround it. Do not show the signature nor contract. You must not use 'assume' nor
- 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPOn9XNMnbxXKTmA3ocQ1CJONqxR

Model: gpt-5.5-2026-04-23

Created at: 1780668077

Finish reason: stop

Usage: 896 tokens in, and 6441 tokens out

```
{
  var n := points.Length;
  area := 0.0;
  var i := 1;

  assert 3 <= n;
  assert points[..] == points[..1] + points[1..];
  assert points[..1] == [points[0]];
  assert [points[0]] + points[1..] == points[..];

  while i < n - 2
    invariant n == points.Length
    invariant 3 <= n
    invariant 1 <= i <= n - 2
    invariant |[points[0]] + points[i..]| >= 3
    invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
  {
    var (x1, y1) := points[0];
    var (x2, y2) := points[i];
    var (x3, y3) := points[i + 1];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    ghost var rem := [points[0]] + points[i..];
    assert i + 1 < n;
    assert |rem| == 1 + n - i;
    assert |rem| > 3;
    assert points[i..] == points[i..i + 1] + points[i + 1..];
    assert points[i..i + 1] == [points[i]];
    assert rem == [points[0]] + [points[i]] + points[i + 1..];
    assert rem[0] == points[0];
    assert rem[1] == points[i];
    assert rem[2] == points[i + 1];
    assert rem[2..] == points[i + 1..];
    assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];
    assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
      ↪ 1..]);
    assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
      ↪ polygon_area(points[..]);

    area := area + triangle_area;
    i := i + 1;
  }

  assert i == n - 2;
  assert i + 1 < n;

  var (x1, y1) := points[0];
```

```

var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(59,29): Error: assertion
  ↪ might not hold
|
59 | assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
  ↪ 1..]);
| ^^

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPRLMhRZGDETyErsbDJBRhGH9v4H

Model: gpt-5.5-2026-04-23

Created at: 1780668235

Finish reason: stop

Usage: 1581 tokens in, and 3706 tokens out

```

{
  var n := points.Length;
  area := 0.0;
  var i := 1;

  assert 3 <= n;
  assert points[..] == points[..1] + points[1..];
  assert points[..1] == [points[0]];
  assert [points[0]] + points[1..] == points[..];
}

```

```

while i < n - 2
  invariant n == points.Length
  invariant 3 <= n
  invariant 1 <= i <= n - 2
  invariant |[points[0]] + points[i..]| >= 3
  invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
{
  var (x1, y1) := points[0];
  var (x2, y2) := points[i];
  var (x3, y3) := points[i + 1];
  var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
  var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

  ghost var rem := [points[0]] + points[i..];
  assert i + 1 < n;
  assert |rem| == 1 + n - i;
  assert |rem| > 3;
  assert points[i..] == points[i..i + 1] + points[i + 1..];
  assert points[i..i + 1] == [points[i]];
  assert rem == [points[0]] + [points[i]] + points[i + 1..];
  assert rem[0] == points[0];
  assert rem[1] == points[i];
  assert rem[2] == points[i + 1];
  assert rem[0] == (x1, y1);
  assert rem[1] == (x2, y2);
  assert rem[2] == (x3, y3);
  assert rem[2..] == points[i + 1..];
  assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];

  if sign < 0.0 {
    assert triangle_area == -sign / 2.0;
    assert polygon_area(rem) == -sign / 2.0 + polygon_area([rem[0]] + rem[2..]);
  } else {
    assert triangle_area == sign / 2.0;
    assert polygon_area(rem) == sign / 2.0 + polygon_area([rem[0]] + rem[2..]);
  }
  assert polygon_area(rem) == triangle_area + polygon_area([rem[0]] + rem[2..]);
  assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
    ↪ 1..]);
  assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
    ↪ polygon_area(points[..]);

  area := area + triangle_area;
  i := i + 1;
}

assert i == n - 2;
assert i + 1 < n;

var (x1, y1) := points[0];

```

```

var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];
assert rem[0] == (x1, y1);
assert rem[1] == (x2, y2);
assert rem[2] == (x3, y3);
if sign < 0.0 {
  assert triangle_area == -sign / 2.0;
  assert polygon_area(rem) == -sign / 2.0;
} else {
  assert triangle_area == sign / 2.0;
  assert polygon_area(rem) == sign / 2.0;
}
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(51,4): Error: assertion
  ↪ might not hold
|

```

```

51 | assert points[i..] == points[i..i + 1] + points[i + 1..];
| ~~~~~

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(65,31): Error: assertion
  ↪ might not hold
|

```

```

65 | assert polygon_area(rem) == -sign / 2.0 + polygon_area([rem[0]] + rem[2..]);
| ^^

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(68,31): Error: assertion
  ↪ might not hold
|

```

```

68 | assert polygon_area(rem) == sign / 2.0 + polygon_area([rem[0]] + rem[2..]);
| ^^

```

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(100,29): Error: assertion
  ↪ might not hold
|

```

```

100 | assert polygon_area(rem) == sign / 2.0;

```

| ^^

Dafny program verifier finished with 2 verified, 4 errors

Correct the error in code by altering only the method body. Produce and show only the
→ Dafny body, including the curly braces that surround it. Do not show the signature
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPSxdJOXVbVKTe2zYansjNeKMu1g

Model: gpt-5.5-2026-04-23

Created at: 1780668335

Finish reason: stop

Usage: 3190 tokens in, and 5983 tokens out

```
{
  var n := points.Length;
  area := 0.0;
  var i := 1;

  assert points[..] == [points[0]] + points[1..];

  while i < n - 2
    invariant n == points.Length
    invariant 3 <= n
    invariant 1 <= i <= n - 2
    invariant |[points[0]] + points[i..]| >= 3
    invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
    decreases n - i
  {
    assert 0 <= i < n;
    assert 0 <= i + 1 <= n;

    ghost var tail := points[i..];
    assert |tail| == n - i;
    assert 1 <= |tail|;
    assert tail == tail[..1] + tail[1..];
    assert |tail[..1]| == 1;
    assert tail[..1][0] == points[i];
    assert tail[..1] == [points[i]];
    assert |tail[1..]| == |points[i + 1..]|;
    forall k | 0 <= k < |tail[1..]|
      ensures tail[1..][k] == points[i + 1..][k]
    {
      assert tail[1..][k] == tail[1 + k];
      assert tail[1 + k] == points[i + 1 + k];
      assert points[i + 1..][k] == points[i + 1 + k];
    }
  }
}
```



```

assert tail[1..] == points[i + 1..];
assert points[i..] == [points[i]] + points[i + 1..];

var (x1, y1) := points[0];
var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert rem == [points[0]] + ([points[i]] + points[i + 1..]);
assert |rem| == 1 + n - i;
assert |rem| > 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

assert |rem[2..]| == |points[i + 1..]|;
forall k | 0 <= k < |rem[2..]|
  ensures rem[2..][k] == points[i + 1..][k]
{
  assert rem[2..][k] == rem[2 + k];
  assert rem[2 + k] == ([points[i]] + points[i + 1..])[1 + k];
  assert ([points[i]] + points[i + 1..])[1 + k] == points[i + 1..][k];
}
assert rem[2..] == points[i + 1..];
assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];
assert |[rem[0]] + rem[2..]| >= 3;

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rsign == sign;
assert rtri == triangle_area;
assert |rem| != 3;
assert polygon_area(rem) == rtri + polygon_area([rem[0]] + rem[2..]);
assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
  ↪ 1..]);
assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
  ↪ polygon_area(points[..]);

area := area + triangle_area;
i := i + 1;
assert area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..]);
}

assert i == n - 2;
assert 0 <= i < n;
assert 0 <= i + 1 < n;

```

```

ghost var tail := points[i..];
assert |tail| == 2;
assert tail[0] == points[i];
assert tail[1] == points[i + 1];

var (x1, y1) := points[0];
var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rsign == sign;
assert rtri == triangle_area;
assert polygon_area(rem) == rtri;
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_13_10_recursive_area_polygon_array/program.dfy(94,29): Error: assertion
  ↳ might not hold
|
94 | assert polygon_area(rem) == rtri + polygon_area([rem[0]] + rem[2..]);
|   ^^

```

Dafny program verifier finished with 2 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↳ Dafny body, including the curly braces that surround it. Do not show the signature
 ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnPV7uPMP3rT94P7hstFeTeRRQbOQ

Model: gpt-5.5-2026-04-23

Created at: 1780668469

Finish reason: stop

Usage: 4746 tokens in, and 6100 tokens out

```
{
  var n := points.Length;
  area := 0.0;
  var i := 1;

  assert points[..] == [points[0]] + points[1..];

  while i < n - 2
    invariant n == points.Length
    invariant 3 <= n
    invariant 1 <= i <= n - 2
    invariant |[points[0]] + points[i..]| >= 3
    invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
    decreases n - i
  {
    assert 0 <= i < n;
    assert 0 <= i + 1 <= n;

    ghost var tail := points[i..];
    assert |tail| == n - i;
    assert 1 <= |tail|;
    assert tail == tail[..1] + tail[1..];
    assert |tail[..1]| == 1;
    assert tail[..1][0] == points[i];
    assert tail[..1] == [points[i]];
    assert |tail[1..]| == |points[i + 1..]|;
    forall k | 0 <= k < |tail[1..]|
      ensures tail[1..][k] == points[i + 1..][k]
    {
      assert tail[1..][k] == tail[1 + k];
      assert tail[1 + k] == points[i + 1 + k];
      assert points[i + 1..][k] == points[i + 1 + k];
    }
    assert tail[1..] == points[i + 1..];
    assert points[i..] == [points[i]] + points[i + 1..];

    var (x1, y1) := points[0];
    var (x2, y2) := points[i];
    var (x3, y3) := points[i + 1];
    var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

    ghost var rem := [points[0]] + points[i..];
```

```

assert rem == [points[0]] + ([points[i]] + points[i + 1..]);
assert |rem| == 1 + n - i;
assert |rem| > 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

assert |rem[2..]| == |points[i + 1..]|;
forall k | 0 <= k < |rem[2..]|
  ensures rem[2..][k] == points[i + 1..][k]
{
  assert rem[2..][k] == rem[2 + k];
  assert rem[2 + k] == ([points[i]] + points[i + 1..])[1 + k];
  assert ([points[i]] + points[i + 1..])[1 + k] == points[i + 1..][k];
}
assert rem[2..] == points[i + 1..];
assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];
assert |[rem[0]] + rem[2..]| >= 3;

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rem[0] == (rx1, ry1);
assert rem[1] == (rx2, ry2);
assert rem[2] == (rx3, ry3);
assert rsign == sign;
assert rtri == triangle_area;
assert |rem| != 3;
assert polygon_area(rem) == (if |rem| == 3 then rtri else rtri + polygon_area([rem[0]]
  ↪ + rem[2..]));
assert polygon_area(rem) == rtri + polygon_area([rem[0]] + rem[2..]);
assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
  ↪ 1..]);
assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
  ↪ polygon_area(points[..]);

area := area + triangle_area;
i := i + 1;
assert area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..]);
}

assert i == n - 2;
assert 0 <= i < n;
assert 0 <= i + 1 < n;

ghost var tail := points[i..];
assert |tail| == 2;
assert tail[0] == points[i];
assert tail[1] == points[i + 1];

```

```

var (x1, y1) := points[0];
var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rem[0] == (rx1, ry1);
assert rem[1] == (rx2, ry2);
assert rem[2] == (rx3, ry3);
assert rsign == sign;
assert rtri == triangle_area;
assert polygon_area(rem) == (if |rem| == 3 then rtri else rtri + polygon_area([rem[0]]
    ↪ + rem[2..]));
assert polygon_area(rem) == rtri;
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Final Program

```

function polygon_area(points: seq<(real, real)>): real
  decreases |points|
  requires |points| >= 3
  ensures polygon_area(points) >= 0.0
{
  if |points| == 3 then
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real
  else
    var (x1, y1) := points[0];
    var (x2, y2) := points[1];
    var (x3, y3) := points[2];
    var sign : real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
    var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0 as real;

```

```

    triangle_area + polygon_area([points[0]] + points[2..])
}

method p_13_10_recursive_area_polygon_array(points: array<(real, real)>) returns (area:
    ↪ real)
    requires points.Length >= 3
    ensures area == polygon_area(points[..])
{
    var n := points.Length;
    area := 0.0;
    var i := 1;

    assert points[..] == [points[0]] + points[1..];

    while i < n - 2
        invariant n == points.Length
        invariant 3 <= n
        invariant 1 <= i <= n - 2
        invariant |[points[0]] + points[i..]| >= 3
        invariant area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..])
        decreases n - i
    {
        assert 0 <= i < n;
        assert 0 <= i + 1 <= n;

        ghost var tail := points[i..];
        assert |tail| == n - i;
        assert 1 <= |tail|;
        assert tail == tail[..1] + tail[1..];
        assert |tail[..1]| == 1;
        assert tail[..1][0] == points[i];
        assert tail[..1] == [points[i]];
        assert |tail[1..]| == |points[i + 1..]|;
        forall k | 0 <= k < |tail[1..]|
            ensures tail[1..][k] == points[i + 1..][k]
        {
            assert tail[1..][k] == tail[1 + k];
            assert tail[1 + k] == points[i + 1 + k];
            assert points[i + 1..][k] == points[i + 1 + k];
        }
        assert tail[1..] == points[i + 1..];
        assert points[i..] == [points[i]] + points[i + 1..];

        var (x1, y1) := points[0];
        var (x2, y2) := points[i];
        var (x3, y3) := points[i + 1];
        var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
        var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

        ghost var rem := [points[0]] + points[i..];
        assert rem == [points[0]] + ([points[i]] + points[i + 1..]);
    }
}

```

```

assert |rem| == 1 + n - i;
assert |rem| > 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

assert |rem[2..]| == |points[i + 1..]|;
forall k | 0 <= k < |rem[2..]|
  ensures rem[2..][k] == points[i + 1..][k]
{
  assert rem[2..][k] == rem[2 + k];
  assert rem[2 + k] == ([points[i]] + points[i + 1..])[1 + k];
  assert ([points[i]] + points[i + 1..])[1 + k] == points[i + 1..][k];
}
assert rem[2..] == points[i + 1..];
assert [rem[0]] + rem[2..] == [points[0]] + points[i + 1..];
assert |[rem[0]] + rem[2..]| >= 3;

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rem[0] == (rx1, ry1);
assert rem[1] == (rx2, ry2);
assert rem[2] == (rx3, ry3);
assert rsign == sign;
assert rtri == triangle_area;
assert |rem| != 3;
assert polygon_area(rem) == (if |rem| == 3 then rtri else rtri + polygon_area([rem[0]]
  ↪ + rem[2..]));
assert polygon_area(rem) == rtri + polygon_area([rem[0]] + rem[2..]);
assert polygon_area(rem) == triangle_area + polygon_area([points[0]] + points[i +
  ↪ 1..]);
assert area + triangle_area + polygon_area([points[0]] + points[i + 1..]) ==
  ↪ polygon_area(points[..]);

area := area + triangle_area;
i := i + 1;
assert area + polygon_area([points[0]] + points[i..]) == polygon_area(points[..]);
}

assert i == n - 2;
assert 0 <= i < n;
assert 0 <= i + 1 < n;

ghost var tail := points[i..];
assert |tail| == 2;
assert tail[0] == points[i];
assert tail[1] == points[i + 1];

```

```

var (x1, y1) := points[0];
var (x2, y2) := points[i];
var (x3, y3) := points[i + 1];
var sign: real := x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2);
var triangle_area := if sign < 0.0 then -sign / 2.0 else sign / 2.0;

ghost var rem := [points[0]] + points[i..];
assert |rem| == 3;
assert rem[0] == points[0];
assert rem[1] == points[i];
assert rem[2] == points[i + 1];

ghost var (rx1, ry1) := rem[0];
ghost var (rx2, ry2) := rem[1];
ghost var (rx3, ry3) := rem[2];
ghost var rsign: real := rx1 * (ry2 - ry3) + rx2 * (ry3 - ry1) + rx3 * (ry1 - ry2);
ghost var rtri := if rsign < 0.0 then -rsign / 2.0 else rsign / 2.0;
assert rem[0] == (rx1, ry1);
assert rem[1] == (rx2, ry2);
assert rem[2] == (rx3, ry3);
assert rsign == sign;
assert rtri == triangle_area;
assert polygon_area(rem) == (if |rem| == 3 then rtri else rtri + polygon_area([rem[0]]
    ↪ + rem[2..]));
assert polygon_area(rem) == rtri;
assert polygon_area(rem) == triangle_area;
assert area + triangle_area == polygon_area(points[..]);
area := area + triangle_area;
}

```

Total Token Usage

Input tokens: 10413

Output tokens: 22230

Reasoning tokens: 16761

Sum of 'total tokens': 32643

Experiment Timings

Overall Experiment started at 1780668076607, ended at 1780668643967, lasting 567360ms (567.36 seconds)

Iteration #4 started at 1780668468671, ended at 1780668643966, lasting 175295ms (175.30 seconds)

Iteration #1 started at 1780668076610, ended at 1780668234586, lasting 157976ms (157.98 seconds)

Iteration #2 started at 1780668234669, ended at 1780668335070, lasting 100401ms (100.40 seconds)

Iteration #3 started at 1780668335071, ended at 1780668468671, lasting 133600ms (133.60 seconds)